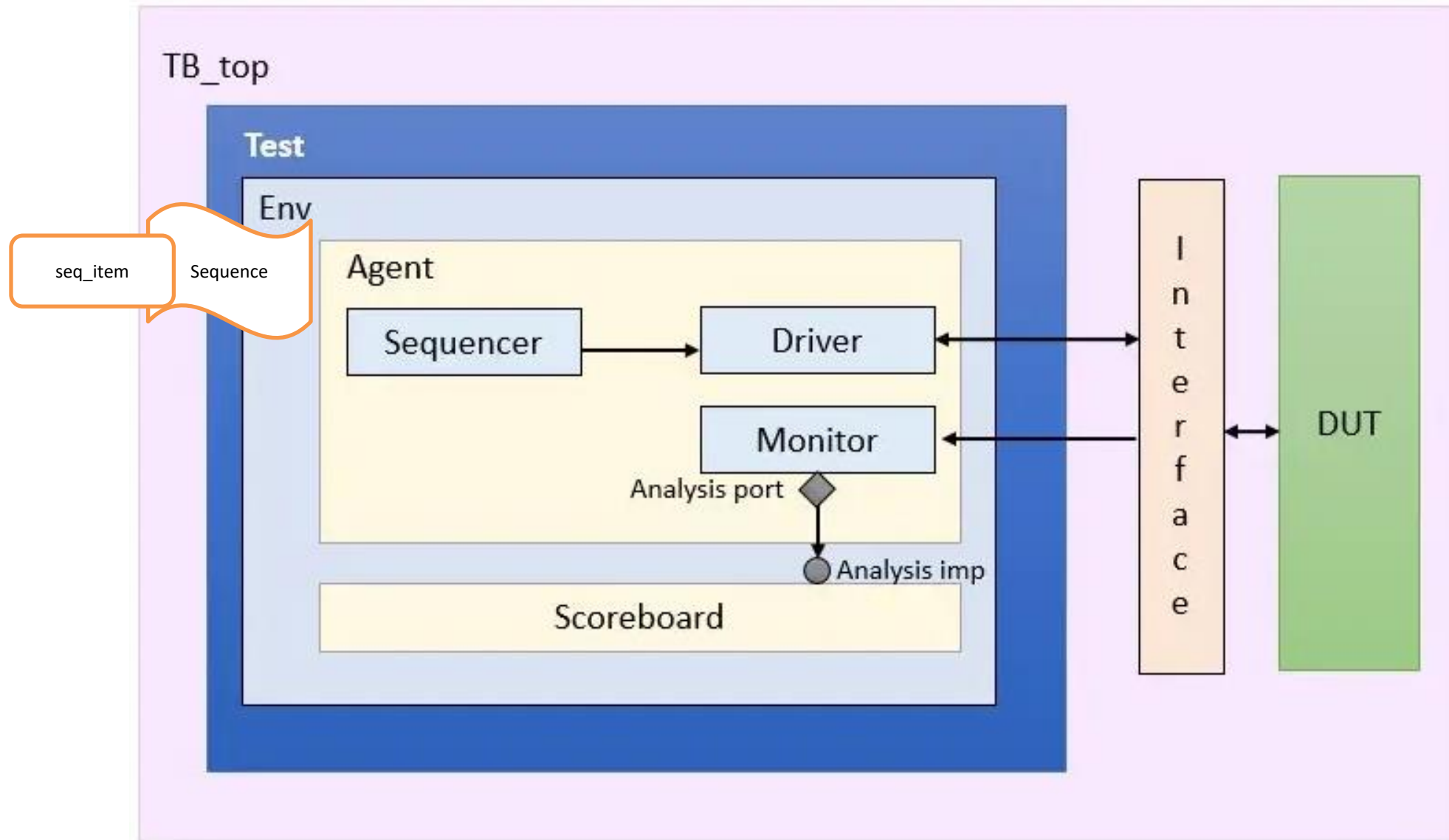


UVM Scoreboard, Environment and Test

Yasir Farooq

Recall the UVM based Testbench



UVM Scoreboard

- This class is used to perform checks, to check whether the DUT responded in the expected way or not after applying respective stimulus
- UVM provides the base class **uvm_scoreboard** extended from the `uvm_component`
- Scoreboard receives the DUT response from monitor via `analysis_port`
 - Therefore, there must be implementation of write method
- Afterwards this received data is compared with the expected data from reference model
 - Test pass or fail decisions are made
 - Usually this is done the `run_phase` of scoreboard

Steps to create Scoreboard

- Create a custom class inherited from `uvm_scoreboard`, register with factory and call function `new`
- Add necessary TLM exports/TLM imp to receive transactions from other components and instantiate them in `build_phase`
- Define the action to be taken when data is received from the analysis port
 - Implementation of `write()`
- Perform checks
- Connect Analysis ports of scoreboard with other components in the environment

UVM Scoreboard

```
1 class scoreboard extends uvm_scoreboard;
2   uvm_analysis_imp #(seq_item, scoreboard) item_collect_export;
3   seq_item item_q[$];
4   `uvm_component_utils(scoreboard)
5
6   function new(string name = "scoreboard", uvm_component parent = null);
7     super.new(name, parent);
8     item_collect_export = new("item_collect_export", this);
9   endfunction
10
11   function void build_phase(uvm_phase phase);
12     super.build_phase(phase);
13   endfunction
14
15   function void write(seq_item req);
16     `uvm_info(get_type_name, $sformatf("Received transaction = %s", req), UVM_LOW);
17     item_q.push_back(req);
18   endfunction
19
20   task run_phase (uvm_phase phase);
21     seq_item sb_item;
22     forever begin
23       wait(item_q.size > 0);
24
25       if(item_q.size > 0) begin
26         sb_item = item_q.pop_front();
27         // Checking comparing logic
28         ...
29       end
30     end
31   endtask
32 endclass
```

UVM Components- Environment

- The environment class defines the testbench structure. It instantiates the component objects and then connect them hierarchically
- Defining an Environment
 - Extend environment with `uvm_env` class
 - `uvm_env` is derived from `uvm_component`
 - Use ``uvm_component_util(<class_name>)` to register environment class with factory
 - Make object of the classes being instantiated in environment
 - Declare `build_phase()` and instantiate the required components
 - `<object> = <class_name>::type_id::create("<object>",this)` is used for this purpose
 - Declare `connect_phase()` to connect instantiated components hierarchically
 - `<component>.<port_in_component>.connect(<component>.<export_in_component>)` is used to connect port of one component to export of other

UVM Environment

```
class env extends uvm_env;
  `uvm_component_utils(env)
  agent agt;
  scoreboard sb;
  func_cov fcov;

  function new(string name = "env", uvm_component parent = null);
    super.new(name, parent);
  endfunction

  function void build_phase(uvm_phase phase);
    super.build_phase(phase);
    agt = agent::type_id::create("agt", this);
    sb = scoreboard::type_id::create("sb", this);
    fcov = func_cov::type_id::create("fcov", this);
  endfunction

  function void connect_phase(uvm_phase phase);
    // connect agent and scoreboard using TLM interface
    // Ex. agt.mon.item_collect_port.connect(sb.item_collect_export);
  endfunction
endclass
```

UVM Components- Test

- **Test is the top level class that instantiates the environment, configures the testbench and initiates construction**
- **Defining an Environment**
 - Extend test class with `uvm_test` class
 - `uvm_test` is derived from `uvm_component`
 - Use ``uvm_component_util(<class_name>)` to register test class with factory
 - Constructor is defined and first line of constructor must be `"super.new(name,parent)"`
 - Operations to perform in test are defined after raising an objection in `run_phase()` using `"phase.raise_objection(this)"`
 - After performing the test, objection is dropped using `"phase.drop_objection(this)"`
 - A specific test is performed by defining test name in top_module in `run_test("<test_name>")` or using `+UVM_TESTNAME` is terminal command

UVM Test

```
class my_test extends uvm_test;
  env env_o;
  base_seq bseq;
  `uvm_component_utils(my_test)

  // constructor
  function new(string name = "my_test", uvm_component parent = null);
    super.new(name, parent);
  endfunction

  function void build_phase(uvm_phase phase);
    super.build_phase(phase);
    env_o = env::type_id::create("env_o", this);
  endfunction

  task run_phase(uvm_phase phase);
    phase.raise_objection(this);
    bseq = base_seq::type_id::create("bseq");

    repeat(10) begin
      #5; bseq.start(env_o.agt.seqr);
    end

    phase.drop_objection(this);
    `uvm_info(get_type_name, "End of testcase", UVM_LOW);
  endtask
endclass
```

Objection Controlling Mechanisms

Objection Controlling Mechanisms

- All components share an objection counter
- There are three methods which controls the objection mechanism:
 - **raise_objection()**
 - Increases the counter by one
 - **drop_objection()**
 - Decreases the counter by one
 - **set_drain_time()**
 - Increases the simulation time by specific number when counter becomes zero
- **+UVM_OBJECTION_TRACE** is a command line option used to trace the objections
 - Log will include information about objections in uvm verification environment

Raise and Drop Objection in UVM

- Objections are used when a particular activity needs to be done before the phase ends, once all of the raised objections for a phase have been dropped, the phase terminates
 - Objections are raised when the activity is started
 - Objections are dropped when the activity is done
- For example, in test class objections can be raised as follows,

```
task main_phase(uvm_phase phase);  
    phase.raise_objection(); //raising objection  
    wr_rd_seq.start(mem_agent.sequencer);  
    phase.drop_objection(); //dropping objection  
endtask
```

Drain time

- **Set_drain_time()** method will drain the object till the specified amount of time. This method is mainly used to extend the simulation time after objection is dropped
- For example, run phase of test class may look like,

```
class mytest extends uvm_test;
...
virtual task run_phase(uvm_phase phase);
    uvm_objection obj;
    obj = phase.get_objection();
    obj.set_drain_time(10ns);

    phase.raise_objection();
    `uvm_do_on(seq, seqr);
    phase.drop_objection();
endtask
endclass
```

Lab Exercise

Lab 9 **Using UVM Scoreboard, Env and Test**

To understand scoreboard, env and test classes in UVM